

RELATÓRIO TÉCNICO: SISTEMA IoT DE CONTROLE TÉRMICO AUTÔNOMO PARA AVICULTURA

Instituição: Universidade de Sorocaba (UNISO)

Disciplina: Fundamentos de IoT

Integrantes do Grupo: Eduardo Augusto

Data: 01 de Outubro de 2026

1. Descrição Geral da Solução

O projeto consiste em um ecossistema IoT autônomo e desacoplado via protocolo **MQTT** para o monitoramento e controle térmico de uma granja avícola. O objetivo principal é prevenir o estresse térmico e a mortalidade das aves por meio da ativação autônoma de um sistema de ventilação sempre que a temperatura do ambiente ultrapassar os limites de segurança.

O sistema foi estruturado em três módulos desacoplados:

- Sensor de Temperatura (Aluno 1 / Publicador):** Simula a leitura de um sensor DHT22 e envia as medições periodicamente em formato JSON.
- Atuador - Ventilador (Aluno 2 / Assinante e Publicador):** Interface gráfica (Tkinter) que escuta os comandos de acionamento ("ON"/"OFF") e responde publicando seu estado de operação.
- Cérebro / Regra de Negócio (Aluno 3 / Nuvem):** Processa os dados de temperatura em tempo real, aplica a lógica condicional e gerencia o envio de comandos para o atuador sem rajadas redundantes de mensagens (spam).

2. Configurações da Rede e Tópicos MQTT

Para garantir a interoperabilidade invisível via internet e evitar interferências no broker público, utilizou-se o prefixo padrão `uniso/granja01/`.

Parâmetro / Tópico	Valor / Descrição
Broker MQTT	<code>broker.emqx.io</code>
Porta	<code>1883</code> (TCP Padrão)

Parâmetro / Tópico	Valor / Descrição
Tópico Sensor	<code>uniso/granja01/sensores/temperatura</code> — O sensor publica periodicamente a temperatura coletada.
Tópico Comando	<code>uniso/granja01/atuadores/ventilador/comando</code> — O Cérebro publica os comandos de atuação (<code>ON</code> / <code>OFF</code>).
Tópico Status	<code>uniso/granja01/atuadores/ventilador/status</code> — O ventilador publica seu estado de execução real (<code>rodando</code> / <code>parado</code>).

3. Decisões Técnicas e Níveis de Qualidade de Serviço (QoS)

As escolhas dos níveis de QoS foram justificadas com base nos requisitos de negócio e latência do sistema:

3.1. Sensor de Temperatura → QoS 0 (At most once)

O sensor publica dados continuamente a cada 5 segundos. Em sistemas de monitoramento contínuo de alta frequência, a perda eventual de um único pacote de dados não compromete a segurança das aves, pois o pacote do intervalo seguinte (5 segundos depois) compensará a perda. O QoS 0 minimiza o overhead de rede, economiza largura de banda e reduz o consumo de energia/bateria dos dispositivos de borda.

3.2. Cérebro e Atuador (Comandos e Status) → QoS 1 (At least once)

O envio do comando de ativação do ventilador é um evento crítico para a preservação da vida animal. Por isso, o uso do **QoS 1** garante que a mensagem de comando (`ON` / `OFF`) chegue com certeza ao atuador. Como o atuador trabalha com comandos de estado (idempotentes), o recebimento duplicado da mensagem "ON" não causa nenhum dano mecânico nem falha lógica no ventilador.

4. Códigos-Fonte do Sistema

4.1. Código do Sensor (`sensor.py`)

```
import paho.mqtt.client as mqtt
```

```
import json
```

```
import random

import time

BROKER = "broker.emqx.io"

PORTA = 1883

TOPICO = "uniso/granja01/sensores/temperatura"

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

def ler_sensor():

    return round(random.uniform(20, 40), 1)

def criar_payload(temperatura):

    return json.dumps({"temperatura": temperatura, "unidade": "C"})

client.connect(BROKER, PORTA)

client.loop_start()

while True:

    temperatura = ler_sensor()

    payload = criar_payload(temperatura)

    client.publish(TOPICO, payload, qos=0)

    print(f"Temperatura publicada: {payload}")

    time.sleep(5)
```

4.2. Código do Ventilador (**ventilador.py**)

```
import json

import tkinter as tk

import paho.mqtt.client as mqtt
```

```
BROKER = "broker.emqx.io"
```

```
PORTA = 1883
```

```
TOPICO_COMANDO = "uniso/granja01/atuadores/ventilador/comando"
```

```
TOPICO_STATUS = "uniso/granja01/atuadores/ventilador/status"
```

```
ligado = False
```

```
def on_connect(client, userdata, flags, rc, props):
```

```
    client.subscribe(TOPICO_COMANDO, qos=1)
```

```
def on_message(client, userdata, msg):
```

```
    global ligado
```

```
    texto = msg.payload.decode().strip()
```

```
    try:
```

```
        dados = json.loads(texto)
```

```
        if isinstance(dados, dict):
```

```
            texto = str(dados.get("comando", ""))
```

```
    except json.JSONDecodeError:
```

```
        pass
```

```
    comando = texto.strip('').upper()
```

```
    if comando == "ON":
```

```
        ligado = True
```

```
        client.publish(TOPICO_STATUS, json.dumps({"status": "rodando"}), qos=1)
```

```
    elif comando == "OFF":
```

```
        ligado = False
```

```

        client.publish(TOPICO_STATUS, json.dumps({"status": "parado"}), qos=1)

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

client.on_connect = on_connect

client.on_message = on_message

client.connect_async(BROKER, PORTA)

client.loop_start()

# Visualização Gráfica Tkinter

janela = tk.Tk()

janela.title("Atuador - Ventilador")

texto = tk.Label(janela, text="", font=("Arial", 24, "bold"), width=20, height=5)

texto.pack()

def atualizar():

    if ligado:

        texto.config(text="VENTILADOR\nLIGADO", bg="green", fg="white")

    else:

        texto.config(text="VENTILADOR\nDESLIGADO", bg="red", fg="white")

    janela.after(200, atualizar)

atualizar()

janela.mainloop()

```

4.3. Código do Cérebro / Regra de Negócio (cerebro.py)

```

n
import json
import paho.mqtt.client as mqtt

```

```
BROKER = "broker.emqx.io"
```

```
PORTA = 1883
```

```
TOPICO_SENSOR = "uniso/granja01/sensores/temperatura"
```

```
TOPICO_COMANDO = "uniso/granja01/atuadores/ventilador/comando"
```

Controle de estado para evitar spam de mensagens

```
estado_ventilador = None
```

```
def on_connect(client, userdata, flags, rc, props):
```

```
    print("Cérebro conectado ao Broker. Aguardando leitura dos sensores...")
```

```
    client.subscribe(TOPICO_SENSOR, qos=0)
```

```
def on_message(client, userdata, msg):
```

```
    global estado_ventilador
```

```
    try:
```

```
        dados = json.loads(msg.payload.decode())
```

```
        temp = dados.get("temperatura")
```

```
        print(f"[CÉREBRO] Temperatura recebida: {temp}°C") # Regra de Negócio: Liga se > 30°C,  
        Desliga se < 26°C
```

```
        if temp > 30.0 and estado_ventilador != "ON":
```

```
            print(f"[ALERTA DE RISCO] Temperatura {temp}°C > 30°C! Enviando comando ON...")
```

```
            client.publish(TOPICO_COMANDO, json.dumps({"comando": "ON"}), qos=1)
```

```
            estado_ventilador = "ON"
```

```
        elif temp < 26.0 and estado_ventilador != "OFF":
```

```
            print(f"[CONDIÇÃO NORMAL] Temperatura {temp}°C < 26°C. Enviando comando OFF...")
```

```
            client.publish(TOPICO_COMANDO, json.dumps({"comando": "OFF"}), qos=1)
```

```
            estado_ventilador = "OFF"
```

```
except Exception as e:
```

```
    print(f"Erro ao processar mensagem: {e}")
```

```
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message
client.connect(BROKER, PORTA)
client.loop_forever()
```

5. Provas Visuais de Funcionamento e Execução em Tempo Real

O fluxo de dados e execução do cenário de risco operacional foi validado conforme a sequência temporal:

1. **Cenário de Risco (Terminal do Sensor):**

O script `sensor.py` simula e publica a medição crítica:

```
`Temperatura publicada: {"temperatura": 33.4, "unidade": "C"}`
```

2. **A Decisão (Terminal do Cérebro):**

O Cérebro capta a mensagem no tópico `uniso/granja01/sensores/temperatura`:

```
`[CÉREBRO] Temperatura recebida: 33.4°C`
```

```
`[ALERTA DE RISCO] Temperatura 33.4°C > 30°C! Enviando comando ON...`
```

O comando em JSON `{"comando": "ON"}` é publicado no tópico de atuadores.

3. **A Salvação (Tela do Atuador - Ventilador):**

A interface Tkinter recebe a mensagem no tópico
`uniso/granja01/atuadores/ventilador/comando`.

A tela muda seu visual para **Fundo Verde** com o texto **"VENTILADOR LIGADO"**.

O atuador publica no tópico de status para confirmar a operação:

```
`{"status": "rodando"}`
```

6. Conclusão

Através do desacoplamento fornecido pelo protocolo MQTT, a arquitetura desenvolvida garantiu que o sensor, o cérebro e o atuador trabalhem de forma totalmente autônoma e independente. O tempo de resposta para neutralizar picos de calor foi reduzido para

milissegundos, garantindo a automação ideal para preservar a vida animal no ambiente agrícola.